First-Order Logic

Chapter 8

Outline

- Why FOL?
- Syntax and semantics of FOL
- Using FOL
- Wumpus world in FOL
- Knowledge engineering in FOL

Pros and cons of propositional logic

- ☺ Propositional logic is declarative
- ☺ Propositional logic allows partial/disjunctive/negated information
  - (unlike most data structures and databases)
- ☺ Propositional logic is compositional:
  - meaning of $B_{1,1} \wedge P_{1,2}$ is derived from meaning of $B_{1,1}$ and of $P_{1,2}$
- ☺ Meaning in propositional logic is context-independent

(unlike natural language, where meaning depends on context)

- ☹ Propositional logic has very limited expressive power

(unlike natural language)

  - E.g., cannot say "pits cause breezes in adjacent squares“
    - except by writing one sentence for each square

Propositional logic is a weak language

- Hard to identify “individuals” (e.g., Mary, 3)
- Can’t directly talk about properties of individuals or relations between individuals (e.g., “Bill is tall”)
- Generalizations, patterns, regularities can’t easily be represented (e.g., “all triangles have 3 sides”)

- First-Order Logic (abbreviated FOL or FOPC) is expressive enough to concisely represent this kind of information

FOL adds relations, variables, and quantifiers, e.g.,

- *"Every elephant is gray"*: $\forall$ x (elephant(x) $\rightarrow$ gray(x))
- *"There is a white alligator"*: $\exists$ x (alligator(X) ^ white(X))

First-order logic

Propositional logic assumes the world contains facts.

First-order logic (like natural language) assumes the world contains

- Objects: people, houses, numbers, colors, baseball games, wars, …
- Relations: red, round, prime, brother of, bigger than, part of, comes between, …
- Functions: father of, best friend, one more than, plus, …

Syntax of FOL: Basic elements

- Constants KingJohn, 2, NUS,...
- Predicates Brother, >,...
- Functions Sqrt, LeftLegOf,...
- Variables x, y, a, b,...
- Connectives $\neg$, $\Rightarrow$, $\wedge$, $\vee$, $\Leftrightarrow$
- Equality =
- Quantifiers $\forall$, $\exists$

Atomic sentences

Atomic sentence = *predicate* (*$term_1$*,...,*$term_n$*)
or *$term_1$* = *$term_2$*

Term = *function* (*$term_1$*,...,*$term_n$*)
or *constant* or *variable*

- E.g., *Brother(KingJohn,RichardTheLionheart)* > *(Length(LeftLegOf(Richard)), Length(LeftLegOf(KingJohn)))*

Complex sentences

- Complex sentences are made from atomic sentences using connectives
-

$$\neg S,\ S_1 \wedge S_2,\ S_1 \vee S_2,\ S_1 \Rightarrow S_2,\ S_1 \Leftrightarrow S_2,$$

E.g.

$Sibling(KingJohn,Richard) \Rightarrow Sibling(Richard,KingJohn)$

$>(1,2) \vee \leq (1,2)$

$>(1,2) \wedge \neg >(1,2)$

Backus–Naur form for FOL

- Sentence → AtomicSentence | ComplexSentence
- AtomicSentence → Predicate | Predicate(Term,...) | Term = Term
- ComplexSentence → (Sentence) | [Sentence] | ¬ Sentence | Sentence ∧ Sentence | Sentence ∨ Sentence | Sentence ⇒ Sentence | Sentence ⇔ Sentence | Quantifier Variable,... Sentence
- Term → Function(Term,...) | Constant | Variable
- Quantifier → ∀| ∃
- Constant → A | X1 | John | ···
- Variable → a | x | s | ···
- Predicate → True | False | After | Loves | Raining | ···
- Function → Mother | LeftLeg | ···
- OPERATOR PRECEDENCE : ¬, =, ∧, ∨, ⇒, ⇔

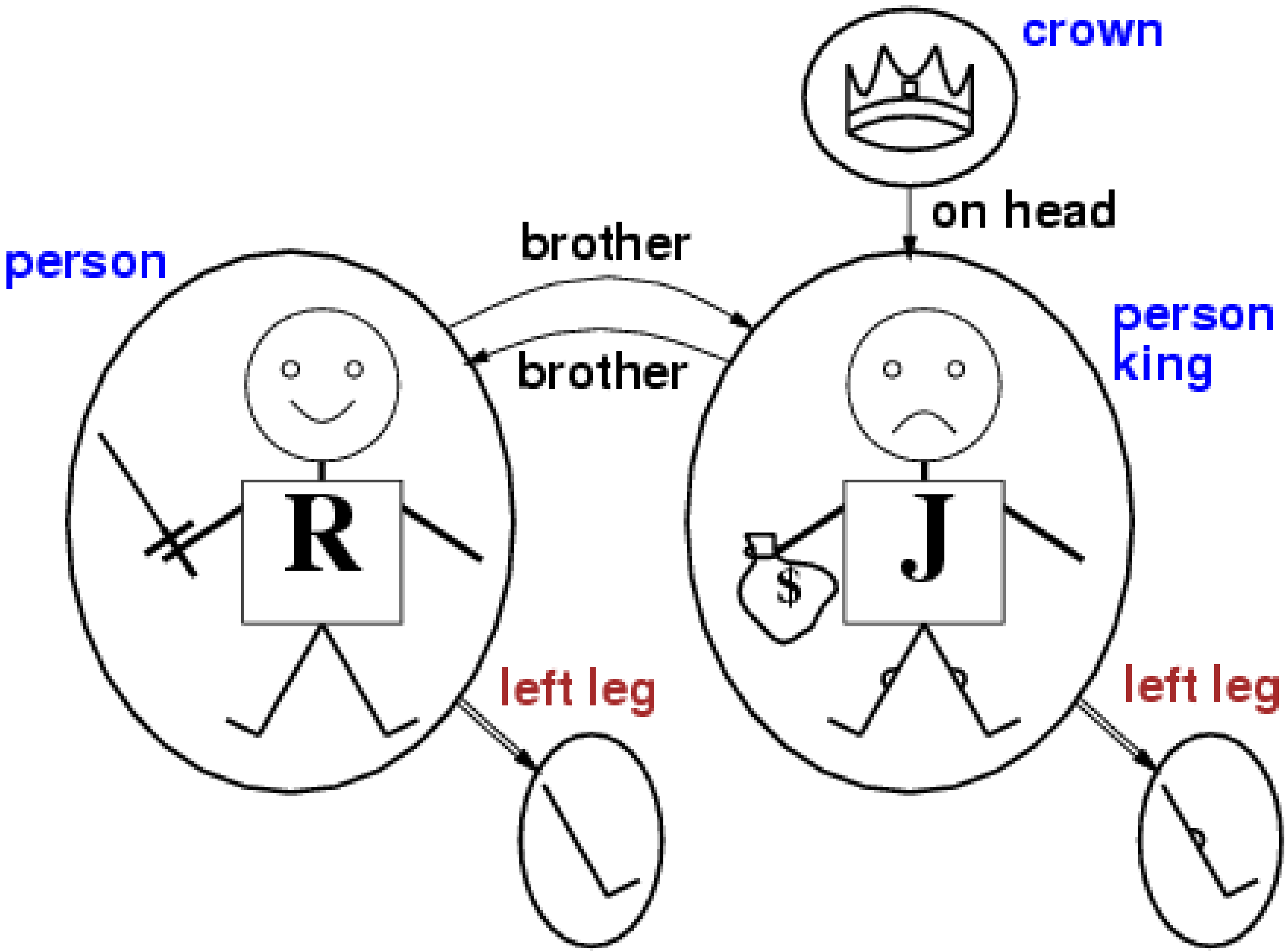
Truth in first-order logic

- Sentences are true with respect to a model and an interpretation
- Model contains objects (domain elements) and relations among them
-
- Interpretation specifies referents for

constant symbols	→	objects
predicate symbols	→	relations
function symbols	→	functional relations

- An atomic sentence *predicate(term*$_1$*,...,term*$_n$*)* is true iff the objects referred to by *term*$_1$*,...,term*$_n$ are in the relation referred to by *predicate*

Models for FOL: Example



Translating English to FOL

Every gardener likes the sun.

$\forall x$ gardener(x) $\rightarrow$ likes(x,Sun)

You can fool some of the people all of the time.

$\exists x\ \forall t$ person(x) $\wedge$time(t) $\rightarrow$ can-fool(x,t)

You can fool all of the people some of the time.

$\forall x\ \exists t$ (person(x) $\rightarrow$ time(t) $\wedge$can-fool(x,t))

$\forall x$ (person(x) $\rightarrow$ $\exists t$ (time(t) $\wedge$can-fool(x,t)))

Equivalent

All purple mushrooms are poisonous.

$\forall x$ (mushroom(x) $\wedge$ purple(x)) $\rightarrow$ poisonous(x)

No purple mushroom is poisonous.

$\neg\exists x$ purple(x) $\wedge$ mushroom(x) $\wedge$ poisonous(x)

$\forall x$ (mushroom(x) $\wedge$ purple(x)) $\rightarrow$ $\neg$poisonous(x)

Equivalent

There are exactly two purple mushrooms.

$\exists x\ \exists y$ mushroom(x) $\wedge$ purple(x) $\wedge$ mushroom(y) $\wedge$ purple(y) ^ $\neg$(x=y) $\wedge$ $\forall z$ (mushroom(z) $\wedge$ purple(z)) $\rightarrow$ ((x=z) $\vee$ (y=z))

Clinton is not tall.

$\neg$tall(Clinton)

X is above Y iff X is on directly on top of Y or there is a pile of one or more other objects directly on top of one another starting with X and ending with Y.

$\forall x\ \forall y$ above(x,y) $\leftrightarrow$ (on(x,y) $\vee$ $\exists z$ (on(x,z) $\wedge$ above(z,y)))

Universal quantification

- $\forall$*<variables>* *<sentence>*

Everyone at NUS is smart:
$\forall x\ At(x,NUS) \Rightarrow Smart(x)$

- $\forall x\ P$ is true in a model *m* iff *P* is true with *x* being each possible object in the model
- Roughly speaking, equivalent to the conjunction of instantiations of *P*
-

$$
\begin{aligned}
& At(KingJohn,NUS) \Rightarrow Smart(KingJohn) \\
\wedge\ & At(Richard,NUS) \Rightarrow Smart(Richard) \\
\wedge\ & At(NUS,NUS) \Rightarrow Smart(NUS) \\
\wedge\ & \ldots
\end{aligned}
$$

A common mistake to avoid

- Typically, $\Rightarrow$ is the main connective with $\forall$
- Common mistake: using $\wedge$ as the main connective with $\forall$:

  $\forall x\ At(x,NUS) \wedge Smart(x)$ means "Everyone is at NUS and everyone is smart"

Existential quantification

- $\exists$<*variables*> <*sentence*>

- Someone at NUS is smart:
- $\exists x$ At(x,NUS) $\wedge$ Smart(x)

- $\exists x\ P$ is true in a model *m* iff *P* is true with *x* being some possible object in the model

- Roughly speaking, equivalent to the disjunction of instantiations of *P*

$$
\begin{aligned}
& \text{At(KingJohn,NUS)} \wedge \text{Smart(KingJohn)} \\
\vee\ & \text{At(Richard,NUS)} \wedge \text{Smart(Richard)} \\
\vee\ & \text{At(NUS,NUS)} \wedge \text{Smart(NUS)}
\end{aligned}
$$

Another common mistake to avoid

- Typically, $\wedge$ is the main connective with $\exists$
- Common mistake: using $\Rightarrow$ as the main connective with $\exists$:
-

$$\exists x \; At(x,NUS) \Rightarrow Smart(x)$$

is true if there is anyone who is not at NUS!

Properties of quantifiers

- $\forall x\ \forall y$ is the same as $\forall y\ \forall x$
- $\exists x\ \exists y$ is the same as $\exists y\ \exists x$
- $\exists x\ \forall y$ is not the same as $\forall y\ \exists x$
- $\exists x\ \forall y$ Loves(x,y)
  - "There is a person who loves everyone in the world"
- $\forall y\ \exists x$ Loves(x,y)
  - "Everyone in the world is loved by at least one person"
- Quantifier duality: each can be expressed using the other
- $\forall x$ Likes(x,IceCream) $\qquad \neg\exists x\ \neg$Likes(x,IceCream)
- $\exists x$ Likes(x,Broccoli) $\qquad \neg\forall x\ \neg$Likes(x,Broccoli)

Equality

- *$term_1$* = *$term_2$* is true under a given interpretation if and only if *$term_1$* and *$term_2$* refer to the same object
-
- E.g., definition of *Sibling* in terms of *Parent*:
-

$$\forall x,y\ Sibling(x,y) \Leftrightarrow [\neg(x = y) \wedge \exists m,f \neg (m = f) \wedge Parent(m,x) \wedge Parent(f,x) \wedge Parent(m,y) \wedge Parent(f,y)]$$

The kinship domain

- One's mother is one's female parent:

$\forall$ m, c Mother (c) = m $\Leftrightarrow$ Female(m) $\wedge$ Parent(m, c) .

- One's husband is one's male spouse:

$\forall$ w, h Husband(h, w) $\Leftrightarrow$ Male(h) $\wedge$ Spouse(h, w) .

- Male and female are disjoint categories:

$\forall$ x Male(x) $\Leftrightarrow$ $\neg$Female(x) .

- Parent and child are inverse relations:

$\forall$ p, c Parent(p, c) $\Leftrightarrow$ Child(c, p) .

- A grandparent is a parent of one's parent:

$\forall$ g, c Grandparent(g, c) $\Leftrightarrow$ $\exists$ p Parent(g, p) $\wedge$ Parent(p, c) .
A sibling is another child of one's parents:

$\forall$ x, y Sibling(x, y) $\Leftrightarrow$ x = y $\wedge$ $\exists$ p Parent(p, x) $\wedge$ Parent(p, y)
.

Sets

1. The only sets are the empty set and those made by adjoining something to a set:

$\forall$ s Set(s) $\Leftrightarrow$ (s = { }) $\vee$ ($\exists$ x, s2 Set(s2) $\wedge$ s = {x|s2}) .

2. The empty set has no elements adjoined into it. In other words, there is no way to decompose { } into a smaller set and an element:

$\neg\exists$ x, s {x|s} = { } .

3. Adjoining an element already in the set has no effect:

$\forall$ x, s x$\in$ s $\Leftrightarrow$ s = {x|s} .

4. The only members of a set are the elements that were adjoined into it. We express this recursively, saying that x is a member of s if and only if s is equal to some set s2 adjoined with some element y, where either y is the same as x or x is a member of s2:

$\forall$ x, s x$\in$ s $\Leftrightarrow$ $\exists$ y, s2 (s = {y|s2} $\wedge$ (x = y $\vee$ x$\in$ s2)) .

5. A set is a subset of another set if and only if all of the first set's members are members of the second set:

$\forall$ s1, s2 s1 $\subseteq$ s2 $\Leftrightarrow$ ($\forall$ x x$\in$ s1 $\Rightarrow$ x$\in$ s2) .

6. Two sets are equal if and only if each is a subset of the other:

$\forall$ s1, s2 (s1 = s2) $\Leftrightarrow$ (s1 $\subseteq$ s2 $\wedge$ s2 $\subseteq$ s1) .

Assertions and queries in first-order logic

- Sentences are added to a knowledge base using TELL, exactly as in propositional logic. Such sentences are called ASSERTIONS.
- TELL(KB, King(John)) .
- TELL(KB, Person(Richard)) .
- TELL(KB, $\forall$ x King(x) $\Rightarrow$ Person(x)) .

- Questions asked with ASK are called queries or goals
- ASK(KB, Person(John))
- ASK(KB, ∃ x Person(x)) .-returns true not useful
- ASKVARS(KB,Person(x))- : returns{x/John} and {x/Richard}.
- . Such an answer is called a substitution or binding list

Wumpus World-FOL

- Sensed value at t=5

Percept([Stench, Breeze, Glitter , None, None], 5) .

Actions Possible:

- Turn(Right), Turn(Left), Forward, Shoot, Grab, Climb .

Query

ASKVARS(∃ a BestAction(a, 5)) ,

returns {a/Grab}

- $\forall$ t, s, g, m, c Percept([s, Breeze, g, m, c], t) $\Rightarrow$ Breeze(t) ,
- $\forall$ t, s, b, m, c Percept([s, b, Glitter , m, c], t) $\Rightarrow$ Glitter(t) ,
- $\forall$ t Glitter (t) $\Rightarrow$ BestAction(Grab, t) .
- $\forall$ x, y, a, b Adjacent([x, y], [a, b]) $\Leftrightarrow$ (x = a $\wedge$ (y = b − 1 $\vee$ y = b + 1)) $\vee$ (y = b $\wedge$ (x = a − 1 $\vee$ x = a + 1)) .

- At(Agent, s, t) to mean that the agent is at square s at time t.
- $\forall t$ At(Wumpus, [2, 2], t).
- $\forall$ x, s1, s2, t At(x, s1, t) $\wedge$ At(x, s2, t) $\Rightarrow$

s1 = s2 .

$\forall$ s, t At(Agent, s, t) $\wedge$ Breeze(t) $\Rightarrow$ Breezy(s)

$\forall$ s Breezy(s) $\Leftrightarrow$ $\exists$ r Adjacent(r, s) $\wedge$ Pit(r) .

$\forall$ t HaveArrow(t + 1) $\Leftrightarrow$ (HaveArrow(t) $\wedge$ $\neg$Action(Shoot, t)) .

Interacting with FOL KBs

- Suppose a wumpus-world agent is using an FOL KB and perceives a smell and a breeze (but no glitter) at *t=5*:

  `Tell`(KB,Percept([Smell,Breeze,None],5))
  `Ask`(KB,∃a BestAction(a,5))

- I.e., does the KB entail some best action at *t=5*?

- Answer: *Yes*, {*a/Shoot*} ← substitution (binding list)

- Given a sentence *S* and a substitution σ,
- *S*σ denotes the result of plugging σ into *S*; e.g.,
  S = Smarter(x,y)
  σ = {x/Hillary,y/Bill}
  *S*σ = Smarter(Hillary,Bill)

- `Ask`(KB,S) returns some/all σ such that KB ⊨ σ

Knowledge base for the wumpus world

- Perception
  - $\forall t,s,b$ Percept([s,b,Glitter],t) $\Rightarrow$ Glitter(t)
  -

- Reflex
  - $\forall t$ Glitter(t) $\Rightarrow$ BestAction(Grab,t)

Deducing hidden properties

- $\forall x,y,a,b$ *Adjacent*$([x,y],[a,b]) \Leftrightarrow$
  $[a,b] \in \{[x+1,y], [x-1,y],[x,y+1],[x,y-1]\}$

Properties of squares:

- $\forall s,t$ *At*$(\text{Agent},s,t) \wedge \text{Breeze}(t) \Rightarrow \text{Breezy}(s)$

  Squares are breezy near a pit:

  Diagnostic rule---infer cause from effect

  $\forall s\ \text{Breezy}(s) \Rightarrow \exists\{r\}\ \text{Adjacent}(r,s) \wedge \text{Pit}(r)$

  – Causal rule---infer effect from cause

  $\forall r\ \text{Pit}(r) \Rightarrow [\forall s\ \text{Adjacent}(r,s) \Rightarrow \text{Breezy}(s)$

- Reference: Artificial Intelligence A Modern Approach,Third Edition by Stuart Russell and Peter Norvig